

California P&C Broker License LLM Benchmark — Master Plan

Data Sources · Evaluation Datasets · Local Model Selection · RAG vs. Web-Search Strategy

April 2026 — Confidential Working Document

1 Project Goal

The goal of this project is to **prove that any well-configured LLM can pass a California Property & Casualty (P&C) broker-agent licensing exam**, thereby demonstrating that the knowledge required to hold a broker license is learnable and reproducible by machine. We will build a rigorous, contamination-free multiple-choice evaluation dataset, run a matrix of local open-source models through multiple inference techniques (RAG, web search, fine-tuning), and track every result in a canonical README and CSV log.

Success = a local base model scoring above the 70 % passing threshold on the held-out test set without seeing answer keys at inference time.

2 Data Sources — Where to Get the Data

Data falls into two buckets: **RAG / knowledge-base documents** (insurance codes, regulation text, study guides) and **evaluation questions** (MCQ banks you will scrape for the held-out test set). Keep them strictly separate.

2.1 Official Regulatory & Reference Documents (RAG Corpus)

These are authoritative, freely accessible sources that should form the backbone of any RAG system. No copyright issues.

Source	What to grab	URL
California Department of Insurance (CDI)	Outline, candidate handbook, licensing rules	https://www.insurance.ca.gov
PSI Exams — CA Candidate Handbook	Exam blueprint, topic weights, sample Qs	https://candidate.psiexams.com
California Insurance Code (CIC)	Full statutory text — searchable via Leginfo	https://leginfo.legislature.ca.gov
California Code of Regulations Title 10	CDI-specific admin rules for licensees	https://govt.westlaw.com/calregs
CDI Bulletins & Circulars	Enforcement actions, interpretation letters	https://www.insurance.ca.gov/0250-insurers/0300-insurers/0200-bullet
NAIC Model Laws & Glossary	National baseline definitions; useful for cross-state topics	https://content.naic.org/model-laws
FEMA / NFIP Docs	Flood insurance rules relevant to CA P&C	https://www.fema.gov/flood-insurance
ISO (Insurance Services Office) Publications	Standard form explanations where publicly available	https://www.verisk.com/insurance/products/iso/

2.2 Practice Exam / MCQ Websites (Evaluation Dataset)

These sites contain multiple-choice questions mirroring the real CA P&C exam. **Scrape carefully — always check Terms of Service. Store questions without answer keys in your public test split; store answers only in the locked evaluation split.**

Site	Notes	Access	URL
Quizlet — CA P&C Insurance Exam Flashcards	Real-style MCQs with answer keys	Free Community-created	https://quizlet.com/search/ca-property-casualty-insurance-exam
ExamTopics — Insurance Exam Community	Community Q&A threads with discussion on tricky topics	Free (registration) group for difficult questions	https://www.examtopics.com

AgentBrokerTrainingCenter.com	Hundreds of CA P&C practice questions; sample at free / paid	Public / private	https://www.abtrainingcenter.com/california-property-casualty-licensing/
Kaplan Financial Education — QBank	Premium QBank; purchase required but general (not limited) random	Paid (\$99)	https://www.kaplanfinancial.com/california-property-casualty-licensing/
ExamFX (National General)	Full CA P&C prep; practice exams closely mirror PSI format.	Paid	https://www.examfx.com/insurance/california
Study.com — CA P&C Practice Test	Free sample exams; detailed answer rationales	Free / subscription	https://study.com/academy/exam/course/california-property-casualty-licensing/
PrepAgent	Insurance-specific prep; includes CA P&C module with 1000+ questions	Paid	https://www.prepagent.com
Insurance License Exam Prep — Flashcards	MCQs organized by CA topic	Free / paid	https://insurancepro.com/exam-prep/california
Pocket Prep App — P&C Insurance	Mobile MCQ app; API may be reversible for bulk app extraction	Paid app	https://www.pocketprep.com/exams/insurance/
TestPrepReview.com — Insurance	Free MCQ question practice tests per topic; no pay wall.	Free	https://www.testprepreview.com/insurance_practice.htm
VanEd Insurance Courses	CA-specific pre-licensing with embedded quizzes	Paid course	https://www.vaned.com/insurance/california
Xcel Solutions (A Kaplan Company)	Scenario-based CA P&C prep questions; etf and code emphasis	Paid	https://www.xcelsolutions.com/california-property-casualty-licensing/
Reddit r/InsuranceAgent	User-shared exam questions and recalled items. Not official but a variety	Free	https://www.reddit.com/r/InsuranceAgent
GitHub — open insurance QA datasets	Search for insurance MCQ repos; some exist for CPCU / licenses	Free	https://github.com (search: insurance exam questions dataset)

Scraping priority order: Start with Quizlet (free, large) → TestPrepReview (free) → Study.com free samples → ExamTopics community threads → purchase Kaplan QBank for the highest-quality structured questions.

3 Top 10 Local Models to Test

These models can all be run via **Ollama** or **llama.cpp** / **vLLM** on local hardware without API keys. Listed from lightest to heaviest; all support 4-bit quantization (GGUF/GPTQ) to cut VRAM requirements roughly in half.

#	Model	Params	VRAM (Q4)	Strengths for P&C MCQ	Ollama command
1	Phi-3.5 Mini Instruct (Microsoft)	3.8B	~3 GB	Fast, strong reasoning per param, good instruction following	<code>ollama run phi3.5</code>
2	Gemma 2 9B Instruct (Google)	9B	~6 GB	Excellent benchmark scores; great at knowledge recall	<code>ollama run gemma2:9b</code>
3	Mistral 7B Instruct v0.3 (Mistral AI)	7B	~5 GB	Industry workhorse; solid MCQ accuracy, fast	<code>ollama run mistral:7b</code>
4	Qwen2.5 7B Instruct (Alibaba)	7B	~5 GB	Strong on factual QA; 128k context; Apache 2.0 license	<code>ollama run qwen2.5:7b</code>
5	Llama 3.3 8B Instruct (Meta)	8B	~6 GB	Latest Llama; strong general reasoning; wide ecosystem support	<code>ollama run llama3.3:8b</code>
6	DeepSeek-R1 7B (DeepSeek AI)	7B	~5 GB	Chain-of-thought reasoning; good at rule application	<code>ollama run deepseek-r1:7b</code>
7	Qwen2.5 32B Instruct (Alibaba)	32B	~20 GB	Best mid-tier; strong compliance reasoning; long context	<code>ollama run qwen2.5:32b</code>
8	Mistral Small 3.1 (Mistral AI)	22B	~14 GB	Improved function calling; good for agentic workflows	<code>ollama run mistral-small</code>
9	Llama 3.3 70B Instruct (Meta)	70B	~40 GB	Near-frontier quality; highest accuracy baseline locally	<code>ollama run llama3.3:70b</code>
10	Qwen3 30B MoE (Alibaba)	30B MoE	~18 GB	MoE efficiency; 32k context; strong reasoning	<code>ollama run qwen3:30b</code>

Hardware note: Models 1–6 (≤8B) run on a single consumer GPU (8 GB VRAM) or Apple M-series. Models 7–9 require 20–40 GB VRAM or CPU offloading with llama.cpp. Start with models 1–6 to validate the pipeline before scaling up.

4 Methods Per Model — RAG vs. Web Search

For each of the 10 models above, we test **two primary inference augmentation methods** plus a zero-shot baseline, giving us 3 conditions × 10 models = 30 runs minimum. Each run is scored on the full held-out MCQ test set and logged to the results CSV.

Method A — Zero-Shot Baseline (No Augmentation)

Feed the question + four choices directly to the model. Prompt format:

SYSTEM: You are a California P&C; insurance licensing expert. Choose the single best answer. USER: Question: {question} A) {a} B) {b} C) {c} D) {d} Answer with a single letter.

Purpose: establishes the raw knowledge baseline. Expected accuracy: 40–60 % for 7B models, 55–75 % for 70B models.

Method B — RAG (Retrieval-Augmented Generation)

Build a vector store from CDI docs, California Insurance Code text, and licensed prep content. At inference time, retrieve the top-k chunks most relevant to the question and prepend them as context. Recommended stack:

- **Embedding model:** nomic-embed-text (via Ollama) or bge-m3
- **Vector store:** Chroma (local, no server) or FAISS
- **Retrieval:** cosine similarity, top-5 chunks, 512-token windows
- **Re-ranker (optional):** cross-encoder/ms-marco-MiniLM-L-6-v2
- **Framework:** LlamaIndex or LangChain — both support Ollama natively

RAG sub-variants to test for each model:

- B1 — RAG with CDI official docs only
- B2 — RAG with full corpus (CDI + Insurance Code + prep content)
- B3 — RAG + chain-of-thought (CoT) prompt (ask model to reason before answering)

Method C — Web Search Tool (Agentic)

Give the model a web-search tool and let it query the internet before answering. This tests whether an agent with real-time retrieval can compensate for gaps in base knowledge. Implementation approaches:

- **Ollama + LangChain Agent:** bind a DuckDuckGo or SerpAPI search tool to the model
- **Prompt:** 'Search the web for the relevant California insurance regulation, then answer.'
- **Limit:** 2–3 search calls per question to control latency and cost
- **Log:** record which queries were fired and which URLs were retrieved

Important: Web search is slower and less deterministic than RAG. Run it on a 100-question sample first; scale to full set only if latency is acceptable (< 30 s/question).

Method D — Self-Consistency / Majority Vote (Optional Extension)

Run each question through the model 5 times at temperature=0.7, then take the majority vote. Typically gains 2–5 % accuracy over single-pass. Apply to the best-performing model from B/C as a final polish step.

5 Full Experiment Matrix

Each cell = one eval run. Score = % correct on held-out MCQ test set (target $\geq 70\%$).

Model	A: Zero-Shot	B1: RAG (CDI)	B2: RAG (Full)	B3: RAG+CoT	C: Web Search
Phi-3.5 Mini 3.8B	—	—	—	—	—
Gemma 2 9B	—	—	—	—	—
Mistral 7B v0.3	—	—	—	—	—
Qwen2.5 7B	—	—	—	—	—
Llama 3.3 8B	—	—	—	—	—
DeepSeek-R1 7B	—	—	—	—	—
Qwen2.5 32B	—	—	—	—	—
Mistral Small 22B	—	—	—	—	—
Llama 3.3 70B	—	—	—	—	—
Qwen3 30B MoE	—	—	—	—	—

Fill in scores as runs complete. Highlight cells $\geq 70\%$ in green, 60–69% in yellow, $< 60\%$ in red.

6 Step-by-Step Execution Plan

PHASE 1 — Data Collection (Week 1–2)

1. Set up a scraping environment: Python + Playwright or Scrapy.
2. Scrape Quizlet CA P&C; flashcard sets → normalize to JSON: {id, question, choices:[A,B,C,D], answer, topic, source}.
3. Scrape TestPrepReview.com free insurance practice tests (no paywall).
4. Collect Study.com free sample questions.
5. Purchase Kaplan QBank (\$99) — export/screenshot all questions; manually convert top 200 to JSON.
6. Download all CDI exam objectives, PSI handbook, and CIC text for the RAG corpus.
7. Deduplicate questions by fuzzy string matching (threshold: > 90 % similar = duplicate).
8. Split: 70 % train | 10 % dev | 20 % test. Lock the test split — never use for training or prompt tuning.
9. Strip answer keys from the test split; store keys only in a separate, encrypted file.

PHASE 2 — RAG Corpus Build (Week 2)

1. Chunk CDI docs + CIC text at 512 tokens with 64-token overlap.
2. Embed using nomic-embed-text via Ollama (free, local).
3. Store in ChromaDB collection named 'ca_pnc_rag'.
4. Test retrieval quality: query 20 known questions, verify relevant chunks appear in top-5.

PHASE 3 — Baseline Runs (Week 3)

1. Install all 10 models via Ollama (start with models 1–6 on local GPU).
2. Run Method A (zero-shot) on each model, temperature=0, 1 pass.
3. Log results to results.csv with columns: model, method, accuracy, n_questions, timestamp, notes.
4. Update README.md with results table after each run.

PHASE 4 — RAG Runs (Week 3–4)

1. Wire LlamaIndex or LangChain RAG pipeline to each Ollama model.
2. Run B1 (CDI only), B2 (full corpus), B3 (RAG + CoT) for each model.
3. Compare B1 vs B2 to measure value of prep content in corpus.
4. Compare B3 vs B2 to measure CoT prompt benefit.

PHASE 5 — Web Search Runs (Week 4–5)

1. Build a LangChain agent per model with DuckDuckGoSearch tool.
2. Run on 100-question sample first — measure latency and tool usage.
3. If average latency < 45 s/question, scale to full test set.
4. Log search queries fired per question for analysis.

PHASE 6 — Analysis & Reporting (Week 5–6)

1. Identify best model × method combination per accuracy tier.
2. Run self-consistency (×5 samples, majority vote) on top-2 performers.
3. Write up findings in README.md with interpretation.
4. Export final results.csv and generate a summary chart (matplotlib).

7 Logging — README.md & results.csv Format

7.1 results.csv Schema

Every run appends one row. Never delete rows — use 'notes' for retraction.

Column	Type	Description / Example
run_id	str	Auto-incremented: RUN-001, RUN-002 ...
timestamp	ISO 8601	2026-04-29T14:32:00Z
model_name	str	qwen2.5:7b
model_params	str	7B
method	str	zero_shot rag_cdi rag_full rag_cot web_search self_consistency
n_questions	int	150
n_correct	int	108
accuracy_pct	float	72.0
pass_fail	str	PASS (threshold=70%)
avg_latency_s	float	2.3
temperature	float	0.0
rag_corpus	str	cdi_only full none
web_search_tool	str	duckduckgo none
n_search_calls	int	0 or avg per question
hardware	str	RTX 4090 24GB
notes	str	First run; no CoT

7.2 README.md Structure

The README is the living project log. Update it after every run via terminal. Structure:

- **## Goal** — one-paragraph summary (static)
- **## Data Sources** — table of sources + scrape status (■ done / ■ in progress / ■ not started)
- **## Eval Dataset** — row counts per split, last updated date
- **## Model × Method Results** — copy of the matrix from Section 5, filled with actual scores
- **## Key Findings** — bullet points per completed phase
- **## Run Log** — newest-first list of runs with date, model, method, accuracy
- **## Next Steps** — what to run next

Terminal update command after each run:

```
python log_run.py --model qwen2.5:7b --method rag_full --accuracy 72.0 --n 150
```

This script appends to results.csv and auto-rewrites the Results table in README.md.

8 Risks & Controls

Risk	Mitigation
Test-set contamination	Lock test split before any training or prompt tuning; keep answer keys in encrypted file
Copyright from scraped MCQs	Track ToS per source; prefer free/CC content; treat paid Kaplan content as internal only
Model pretraining contamination	Check if exact questions appear verbatim via string search; flag and remove from eval
Hallucinated legal authority	Rubric includes 'no hallucinated statute'; cross-check cited codes against CIC
Short-cut learning on answer format	Shuffle answer-choice order; test with both lettered and unlabeled choices
Web search returning wrong state law	Prompt must specify California jurisdiction; post-filter for CA sources
Latency on 70B models	Run 70B on GPU server only; use Q4_K_M quantization; batch overnight

9 Quick-Start Checklist

- [] Install Ollama on local GPU machine
- [] Pull first 6 models: phi3.5, gemma2:9b, mistral:7b, qwen2.5:7b, llama3.3:8b, deepseek-r1:7b
- [] Scrape Quizlet + TestPrepReview → normalize to JSON → deduplicate
- [] Create train / dev / test splits; encrypt test answer keys
- [] Download CDI exam outline + PSI handbook + California Insurance Code
- [] Build ChromaDB vector store with nomic-embed-text embeddings
- [] Run zero-shot baseline on all 6 small models → populate results.csv
- [] Update README.md with first results
- [] Run RAG (B1, B2, B3) for each model
- [] Run web-search agent on 100-Q sample for each model
- [] Scale web-search to full test if latency < 45 s/Q
- [] Pull + test 70B models on larger hardware
- [] Run self-consistency on top-2 best performers
- [] Write final analysis; export results chart